

b. Run the Docker Container and Test the Flask Application

Aim

To run the Docker container for the Flask application and verify that the application is working correctly.

Prerequisites

Before performing this practical, ensure that:

- Docker Desktop is installed and running.
 - The Flask application is created.
 - The Dockerfile is created.
 - The Docker image has been built successfully.
-

Step 1: Open Docker Desktop

1. Click **Start**.
2. Search for **Docker Desktop**.
3. Open Docker Desktop.
4. Wait until the status shows:

Engine Running

Step 2: Open the Project Folder

Open **Command Prompt**, **PowerShell**, or the **VS Code Terminal**.

Navigate to your project folder:

```
cd C:\Users\YourName\Documents\Flask_Project
```

Replace the path with your actual project folder.

Step 3: Verify the Docker Image

Check whether the Docker image exists.

docker images

Example Output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
flask-app	latest	a1b2c3d4e5f6	5 minutes ago	170MB

If you do not see flask-app, build the image first:

docker build -t flask-app .

Step 4: Run the Docker Container

Run the Flask container:

docker run -d -p 5000:5000 flask-app

Explanation

Command	Description
docker run	Creates and starts a new container
-d	Runs the container in detached (background) mode
-p 5000:5000	Maps port 5000 on your computer to port 5000 inside the container
flask-app	Name of the Docker image

Step 5: Verify the Container

Check whether the container is running:

docker ps

Example Output:

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS
4a2d6c8f1b9e	flask-app	"python app.py"	Up 20 seconds	0.0.0.0:5000->5000/tcp

If the status is **Up**, the application is running successfully.

Step 6: Test the Application

Open any web browser.

Enter the following URL:

`http://localhost:5000`

Expected Output

Welcome to Dockerized Flask Application

If your application has another route such as /about, test it as well:

`http://localhost:5000/about`

Expected Output:

Docker Practical

Step 7: Test Using the Terminal (Optional)

Instead of using a browser, you can also test the application with:

`curl http://localhost:5000`

Expected Output:

Welcome to Dockerized Flask Application

Step 8: Check Container Logs

If the browser does not display the application, view the logs:

`docker logs <container_id>`

Example:

`docker logs 4a2d6c8f1b9e`

Example Output:

- * Running on all addresses (0.0.0.0)
- * Running on http://127.0.0.1:5000

The logs help identify startup errors or missing dependencies.

Step 9: Stop the Container

First, list the running containers:

```
docker ps
```

Then stop the container:

```
docker stop <container_id>
```

Example:

```
docker stop 4a2d6c8f1b9e
```

Step 10: Restart the Container

List all containers:

```
docker ps -a
```

Restart the stopped container:

```
docker start <container_id>
```

Example:

```
docker start 4a2d6c8f1b9e
```

Open the browser again:

```
http://localhost:5000
```

The application should be accessible again.

Expected Output

- The Docker container starts successfully.
 - The container appears in the output of `docker ps`.
 - The Flask application opens successfully at `http://localhost:5000`.
 - The application responds correctly to all defined routes.
-

Common Errors and Solutions

Error	Cause	Solution
Unable to find image 'flask-app:latest' locally	Docker image has not been built	Run <code>docker build -t flask-app .</code>
Bind for 0.0.0.0:5000 failed: port is already allocated	Port 5000 is already in use	Stop the existing container (<code>docker stop <container_id></code>) or use another port, e.g. <code>docker run -d -p 5001:5000 flask-app</code>
Browser shows Problem loading page	Container is not running or the application crashed	Check <code>docker ps</code> and then <code>docker logs <container_id></code>
No such container	Incorrect container ID	Copy the correct ID from <code>docker ps</code> or <code>docker ps -a</code>
